

Программно-аппаратные комплексы ИИ

Интерактивные презентации по вопросам экзамена (2 семестр) • 5 блоков × 5 вопросов

Презентация 1: Вычислительная база ИИ

Вопрос 1: 1. Роль вычислительных средств в реализации систем ИИ

Конспект:

ИИ-модели требуют обработки огромных объёмов данных. Вычислительные средства определяют скорость обучения, качество инференса и масштабируемость. Современные системы ИИ переходят от CPU к GPU/TPU из-за массового параллелизма матричных операций.

Практическое задание:

Оцените, почему обучение Transformer-модели на CPU займёт в десятки раз больше времени, чем на GPU.

Transformer содержит операции внимания и умножения матриц (GEMM), которые имеют сложность $O(N^2)$. CPU оптимизирован для последовательных операций и ветвлений, а GPU содержит

Вопрос 2: 2. Свойства универсальных вычислителей (CPU) в ИИ

Конспект:

CPU обеспечивает высокую гибкость, поддерживает сложные ветвления, управление памятью и последовательную логику. Подходит для предобработки данных, инференса лёгких моделей и управления рабочим процессом.

Практическое задание:

Определите, при каких сценариях CPU остаётся предпочтительным решением для задач ИИ.

- 1) Работа с разреженными графами и динамическими структурами данных.
- 2) Предобработка текста/аудио, где важны последовательные алгоритмы.
- 3) Инференс на edge-устройствах с ограниченным энергопотреблением.
- 4)

тысячи ядер, способных выполнять матричные операции одновременно. При размере батча 1024 и длине последовательности 512, GPU достигает ускорения в 30–50× за счёт параллелизма и высокой пропускной способности памяти.

Координация распределённого обучения (orchestration).

Вопрос 3: 3. Специализированные вычислители и их отличительные черты

Конспект:

TPU, NPU, VPU и AI-ускорители оптимизированы под тензорные операции, низкую точность (INT8/FP16) и энергоэффективность. Имеют фиксированные конвейеры и минимум логики ветвления.

Практическое задание:

Сравните NPU и GPU для задачи детекции объектов на видеопотоке в реальном времени.

GPU даёт высокую точность и гибкость (поддержка разных фреймворков), но потребляет больше энергии. NPU спроектирован под инференс с фиксированной точностью,

Вопрос 4: 4. Концепция вычислителей с массовым параллелизмом

Конспект:

Основана на модели SIMD/SIMT: одна инструкция выполняется над множеством потоков/данных. Практически реализуется в GPU-кластерах, суперкомпьютерах и распределённых системах обучения.

Практическое задание:

Разработайте концепцию параллельной свёртки массива из 10^8 элементов на GPU.

1) Разбить массив на блоки по 256 потоков. 2) В каждом потоке читать элемент и записывать в локальную память. 3) Использовать tree-based reduction: пары потоков суммируют результаты в общей памяти. 4) Повторять $\log_2(N)$

обеспечивает стабильную задержку < 10 мс и низкое тепловыделение, что критично для embedded-систем и IoT-камер.

шагов. Итог: сложность $O(\log N)$ вместо $O(N)$.

Вопрос 5: 5. Преимущества матричных процессоров для ИИ

Конспект:

Матричные процессоры (TPU, матричные ускорители) используют систолические массивы: данные протекают через фиксированную сетку вычислительных ячеек, минимизируя обращения к памяти.

Практическое задание:

Объясните, почему систолический массив ускоряет умножение матриц по сравнению со скалярным процессором.

В скалярном процессоре каждая операция требует загрузки из памяти и записи результата. В систолическом массиве данные поступают потоком, умножаются и накапливаются внутри массива без промежуточной записи. Пропускная способность ограничена тактовой частотой, а не шиной памяти, что даёт

ускорение в 10–100× для
GEMM.

Презентация 2: Архитектура и программирование GPU

Вопрос 1: 6. Нейроморфные процессоры: архитектура и применение

Конспект:

Моделируют биологические нейронные сети через импульсные нейроны (SNN). Используют событийную логику, асинхронные вычисления и сверхнизкое энергопотребление.

Практическое задание:

Опишите, как нейроморфный чип обрабатывает сигнал от камеры событийного типа (DVS).

DVS генерирует импульсы только при изменении освещённости. Нейроморфный процессор принимает эти импульсы как входные спайки, обрабатывает их через мембранную модель нейрона и формирует выходные спайки. Отсутствие тактового сигнала и статической памяти снижает энергопотребление на 2–3 порядка по сравнению с классическими GPU.

Вопрос 2: 7. Эффективность GPU для параллельных вычислений в ИИ

Конспект:

GPU обладает высокой пропускной способностью памяти (HBM/GDDR6), тысячами вычислительных ядер и специализированными тензорными ядрами. Оптимален для data-parallel задач.

Практическое задание:

Рассчитайте, является ли операция свёртки 3×3 на изображении 4K memory-bound или compute-bound.

Для одного выхода требуется 9 чтений и 1 запись. При FP32 это ~40 байт на 1 операцию с плавающей точкой. Если соотношение байт/операция > 10–15, задача memory-bound. Свёртка обычно memory-bound, поэтому оптимизация достигается через shared memory и свёрточные ядра с минимизацией глобальных обращений.

Вопрос 3: 8. Сравнение OpenCL и CUDA

Конспект:

CUDA – проприетарный стек NVIDIA, максимальная производительность, экосистема библиотек. OpenCL – кроссплатформенный стандарт, работает на AMD/Intel/CPU, но сложнее в отладке и оптимизации.

Практическое задание:

Выберите платформу для лаборатории, где используются GPU NVIDIA и AMD.

Для прототипирования и кросс-совместимости: OpenCL или SYCL. Для production-обучения нейросетей и использования cuDNN/cuBLAS: CUDA. Решение: гибридный стек (OpenCL для универсальных задач, CUDA для ИИ-пайплайнов) или переход на ROCm/SYCL при миграции.

Вопрос 4: 9. Kernel-based программирование

Конспект:

Ядро (kernel) – функция, выполняемая на устройстве. Запускается в иерархии: grid → block → thread. Позволяет явно управлять параллелизмом.

Практическое задание:

Напишите концептуальный CUDA-код для поэлементного сложения двух векторов.

```
__global__ void vecAdd(float* A,
float* B, float* C, int n) {
    int i = blockIdx.x * blockDim.x +
threadIdx.x;
    if (i < n) C[i] = A[i] + B[i];
}
// Запуск:
vecAdd<<<(n+255)/256,
256>>>(d_A, d_B, d_C, n);
Объяснение: Каждый поток
обрабатывает один индекс,
проверка границы
предотвращает выход за
массив.
```

Вопрос 5: 10. Программные стеки параллельных вычислений

Конспект:

Стек включает: драйвер устройства
→ runtime (CUDA/OpenCL) →
библиотеки (cuBLAS, MKL) →
фреймворки (PyTorch, TensorFlow).
Каждый уровень абстрагирует
низкоуровневую оптимизацию.

Практическое задание:

*Проследите путь выполнения
операции `torch.matmul()` до GPU.*

1) PyTorch вызывает ATen-обёртку. 2) ATen формирует дескрипторы тензоров. 3) Вызывается cuBLAS/cuDNN через CUDA driver API. 4) Компилятор PTX → SASS (бинарный код для конкретной архитектуры). 5) Драйвер загружает бинарник в VRAM и запускает kernel.

Презентация 3: Библиотеки и оптимизация

Вопрос 1: 11. Популярные библиотеки для параллельных вычислений

Конспект:

cuBLAS (линейная алгебра), cuDNN (нейросети), OpenMPI (распределённые вычисления), oneMKL (кроссплатформенная математика). Ускоряют типовые операции в 10–100×.

Практическое задание:

Обоснуйте выбор библиотек для обучения ResNet-50 на кластере GPU.

cuDNN для свёрток и пулинга, cuBLAS для полносвязных слоёв, NCCL для AllReduce между узлами, PyTorch Distributed для оркестрации. Такой стек обеспечивает максимальную утилизацию Tensor Cores и минимизирует накладные расходы на синхронизацию.

Вопрос 2: 12. Ускорение ML специализированным оборудованием

Конспект:

ASIC/TPU, FPGA и Tensor Cores позволяют выполнять операции с пониженной точностью (FP16, INT8, BF16) с аппаратным масштабированием и конвейеризацией.

Практическое задание:

Оцените выигрыш от перехода с FP32 на INT8 при инференсе BERT.

INT8 уменьшает объём памяти в 4×, пропускная способность Tensor Cores увеличивается в 2–3×. Точность падает <1% при Quantization-Aware Training. Итог: ускорение инференса в 2.5–4× при снижении энергопотребления на ~30%.

Вопрос 3: 13. Преимущества GPU перед CPU для задач ML

Конспект:

Вопрос 4: 14. Архитектура современных GPU

Конспект:

Массовый параллелизм, высокая пропускная способность памяти, аппаратные матричные ускорители, оптимизированные драйверы и экосистема фреймворков.

Практическое задание:

Проведите мысленный бенчмарк: инференс LSTM vs Transformer на CPU и GPU.

LSTM имеет последовательную зависимость, поэтому ускорение на GPU умеренное (2–5×). Transformer полностью параллелизуем по токенам и головам внимания, ускорение достигает 15–30×. Вывод: GPU наиболее эффективен для архитектур с data-level parallelism.

Состоит из Streaming Multiprocessors (SM), каждый содержит CUDA-ядра, Tensor Cores, shared memory, регистровые файлы. Соединяются через NVLink/HBM2e.

Практическое задание:

Объясните, как SM скрывает задержку доступа к памяти.

Каждый SM поддерживает сотни активных warps. Когда один warp ожидает данные из глобальной памяти, планировщик мгновенно переключается на другой готовый warp. Это latency hiding достигается за счёт избыточного параллелизма и быстрой смены контекста внутри SM.

Вопрос 5: 15. Трудности разработки ПО для GPU

Конспект:

Сложность отладки, управление памятью, синхронизация потоков, оптимизация occupancy, аппаратные ограничения (размер shared memory, лимит регистров).

Практическое задание:

Найдите причину падения производительности при

использовании атомарных операций в kernel.

Атомарные операции (atomicAdd) сериализуют доступ к памяти, превращая параллельный код в последовательный. Решение: использовать приватные аккумуляторы в регистрах/shared memory, а затем выполнять атомарное сложение один раз на блок/поток.

Презентация 4: Управление памятью и SIMT

Вопрос 1: 16. Функции cuDNN и cuBLAS

Конспект:

cuDNN оптимизирует свёртки, пулинг, RNN, нормализацию. cuBLAS реализует BLAS-уровни 1–3 (векторы, матрицы, GEMM). Обе используют Tensor Cores.

Практическое задание:

Покажите, как cuDNN ускоряет прямое распространение свёрточного слоя.

- 1) Выбирает оптимальный алгоритм (имплицитный GEMM, FFT, Winograd) через автопоиск.
- 2) Формирует дескрипторы тензоров.
- 3) Запускает оптимизированный kernel с минимизацией bank conflicts в shared memory.
- 4) Возвращает результат через асинхронные очереди.

Вопрос 2: 17. Распределение нагрузки между ядрами GPU

Конспект:

Зависит от grid/block size, оосурансу, выравнивания по warp (32 потока), использования streams для асинхронности.

Практическое задание:

Подберите размер блока для матричного умножения 4096×4096 .

Оптимально: blockDim = (16, 16) или (32, 8). Оба дают 256 потоков/блок. Кратность 32 гарантирует полный warp. Обеспечивает высокую оосурансу (~75%) и минимизирует регистровое давление.

Вопрос 3: 18. Концепция SIMT и воплощение в GPU

Конспект:

Single Instruction, Multiple Threads: один warp (32 потока) выполняет одну инструкцию. Если потоки

Вопрос 4: 19. Управление памятью в параллельных вычислениях

Конспект:

Иерархия: регистры → shared memory → constant/texture →

ветвятся, происходит serial execution (divergence).

Практическое задание:

Продemonстрируйте влияние branch divergence на производительность.

Код: `if (threadIdx.x < 16) { A = B * 2; } else { A = B + 2; }`

Первые 16 потоков выполнят умножение, остальные ждут. Затем выполнят сложение.

Производительность падает в 2×. Решение: restructuring логики, использование warp-level primitives или маскирование.

global (VRAM). Ключ к оптимизации: coalesced access и тилирование.

Практическое задание:

Оптимизируйте чтение матрицы из глобальной памяти в kernel.

1) Выровнить данные по 128 байт. 2) Использовать shared memory для загрузки подблока (tiling). 3) Читать последовательно (stride=1) для коалесцирования. 4) Избегать bank conflicts через padding. Результат: снижение latency на 60–80%.

Вопрос 5: 20. Повышение эффективности параллельного программирования

Конспект:

Kernel fusion, mixed precision, asynchronous copy, profiling (Nsight), устранение false sharing, использование Tensor Cores.

Практическое задание:

Предложите план профилирования медленного kernel.

1) Запустить Nsight Compute. 2) Проверить: occupancy, memory

throughput, warp stall reasons. 3) Если memory-bound → tiled load, coalescing. 4) Если compute-bound → Tensor Cores, FP16/BF16. 5) Fusion смежных kernel'ов для снижения overhead запуска.

Презентация 5: Системная интеграция и перспективы

Вопрос 1: 21. Взаимодействие CPU и GPU в единой системе

Конспект:

Обмен через PCIe/NVLink. Host управляет памятью, запускает kernels, обрабатывает последовательную логику. Device выполняет массовые вычисления.

Практическое задание:

Спроектируйте pipeline для real-time инференса видеоаналитики.

1) CPU захватывает кадр → pinned memory. 2) Асинхронный cudaMemcpyAsync в VRAM. 3) GPU запускает inference kernel. 4) Результат копируется обратно. 5) CPU пост-обрабатывает и выводит. Использование streams и CUDA events минимизирует простои.

Вопрос 2: 22. Факторы выбора CPU vs GPU для задач ИИ

Конспект:

Зависит от: размера данных, параллелизма, бюджета, энергопотребления, задержки, экосистемы фреймворков.

Практическое задание:

Составьте матрицу выбора для edge-дрона и облачного дата-центра.

Edge-дрон: CPU/NPU (низкое энергопотребление, детерминированная задержка, ограниченная масса). Дата-центр: GPU/TPU кластер (высокая пропускная способность, масштабируемость, обучение больших моделей). Критерии: FLOPS/W, latency, cost/training hour.

Вопрос 3: 23. Ограничения GPU и пути обхода

Конспект:

Вопрос 4: 24. Потенциал FPGA в системах ИИ

Конспект:

Лимит VRAM, PCIe bottleneck, single precision overhead, тепловыделение, сложность отладки.

Практическое задание:

Как решить OOM-ошибку при обучении LLM на одной GPU?

1) Gradient Checkpointing (trade-off memory/compute). 2) Mixed Precision (FP16/BF16). 3) ZeRO-optimization (partition optimizer states). 4) Data/Model Parallelism. 5) Offload в CPU RAM (DeepSpeed). Комбинация снижает потребление VRAM на 60–80%.

Программируемые логические матрицы позволяют создавать кастомные конвейеры, обеспечивают низкую детерминированную задержку, энергоэффективность, аппаратную безопасность.

Практическое задание:

Сравните FPGA и GPU для потоковой обработки финансовых сигналов.

FPGA: задержка <1 мкс, фиксированная пропускная способность, низкое энергопотребление, сложность разработки (HLS/Verilog). GPU: задержка ~0.1–1 мс, гибкость, высокая пиковая производительность, но джиттер из-за ОС и планировщика. Для HFT предпочтительнее FPGA.

Вопрос 5: 25. Перспективные направления развития вычислительной техники для ИИ

Конспект:

Оптические нейросети, 3D-стекирование памяти (HBM4), квантово-классические гибриды, нейроморфные чипы 2-го поколения, фотонные интерконнекты.

Практическое задание:

Оцените влияние оптических вычислений на энергоэффективность ИИ.

Оптические матричные умножения выполняются за счёт интерференции света, потребляя $\sim$ фДж/операцию против $\sim$ пДж у CMOS. Это снижает тепловыделение на 2–3 порядка и позволяет создавать системы с эксафлопсной производительностью без криогенного охлаждения. Основной барьер: точность и интеграция с классической логикой.

Разработано на основе РПД Б1.О.02.02 (Елецкий ГУ им. И.А. Бунина, 2025)
Все материалы предназначены для самоподготовки и закрепления знаний.